

Politechnika Warszawska

Wydział Elektryczny

Instytut Sterowania i Elektroniki Przemysłowej

Sprawozdanie z projektu „Wielowątkowość wysokopoziomowa”



|                   |         |
|-------------------|---------|
| Michał Bobrycki   | 331 766 |
| Karol Cegliński   | 331 773 |
| Przemysław Cichoń | 331 775 |
| Tomasz Rachwał    | 331 796 |
| Kacper Zawadka    | 331 813 |

3 maja 2026

Prowadzący: dr hab. inż. Dominik Olszewski

## Spis treści

|   |                                                 |   |
|---|-------------------------------------------------|---|
| 1 | Cel projektu                                    | 3 |
| 2 | Opis struktury kodu i implementacji             | 4 |
| 3 | Porównanie wykorzystanych metod wielowątkowości | 5 |
| 4 | Analiza wyników i zachowania animacji           | 6 |
| 5 | Analiza z wykorzystaniem profilera              | 7 |
| 6 | Wnioski końcowe                                 | 8 |
|   | Spis rysunków                                   | 9 |

## 1. Cel projektu

Głównym celem projektu było zapoznanie się z podejściem wysokopoziomowym do tworzenia wątków i zarządzania nimi w języku Java. Projekt wymagał zaimplementowania aplikacji graficznej (animacji), w której wiele elementów porusza się równocześnie, oraz przeprowadzenia analizy porównawczej między klasyczną techniką wykorzystującą interfejs `Runnable` i klasę `Thread`, a metodami z pakietu `java.util.concurrent` (`Executor`, `ExecutorService`, `ScheduledExecutorService`).

## 2. Opis struktury kodu i implementacji

Aplikacja została napisana z wykorzystaniem biblioteki Swing i składa się z kilku kluczowych klas i segmentów:

- **ThreadingProject\_P1**: Klasa główna, definiująca parametry brzegowe, takie jak liczba elementów (100) oraz rozmiar puli wątków (4). Uruchamia ona GUI w bezpiecznym wątku `Event Dispatch Thread` za pomocą `SwingUtilities.invokeLater`.
- **Animacja**: Główna klasa zarządzająca oknem (dziedzicząca po `JFrame`). Zawiera metody podpięte pod przyciski interfejsu, z których każda inicjuje ruch obiektów za pomocą innej techniki wielowątkowej. Klasa ta zawiera również metodę `heavyCalculations()`, która za pomocą operacji trygonometrycznych sztucznie obciąża procesor, symulując złożone obliczeniowo zadanie.
- **KrokAnimacji**: Implementacja interfejsu `Runnable`. W metodzie `run()` zaimplementowano pętlę `while`, która wykonuje ciężkie obliczenia, przesuwa klocek w prawo i odświeża interfejs tak długo, aż klocek dotrze do prawej krawędzi okna.
- **PanelAnimacji i RuchomyElement**: Klasy odpowiedzialne za warstwę wizualną. **PanelAnimacji** przechowuje listę klocków (`CopyOnWriteArrayList` dla bezpieczeństwa wątkowego) i odpowiada za ich rysowanie, natomiast **RuchomyElement** to prosta struktura przechowująca współrzędne X i Y.

### 3. Porównanie wykorzystanych metod wielowątkowości

W projekcie wykorzystano cztery podejścia do uruchamiania zadań. Na podstawie analizy teoretycznej oraz budowy poszczególnych interfejsów, można je scharakteryzować następująco:

#### **Runnable + Thread**

Opiera się na ręcznym tworzeniu i uruchamianiu każdego wątku z osobna (`new Thread(...).start()`). System operacyjny musi powołać nowy, fizyczny wątek dla każdego zadania, co przy dużej ich liczbie (w tym przypadku 100) drastycznie obciąża procesor i pamięć ze względu na koszt tzw. przełączania kontekstu. Brakuje tu mechanizmów zarządzania pulą i reużywalności zasobów.

#### **Executor**

Najprostszy interfejs z pakietu `java.util.concurrent`. Oddziela definicję zadania od mechanizmu jego uruchomienia, udostępniając jedynie metodę `execute()`. W implementacji projektu użyto krótkiego zapisu, który w tle inicjuje i uruchamia klasyczny wątek. Jest to interfejs nie dający kontroli nad cyklem życia wątków ani nie pozwalający wprost sprawdzić statusu zakończenia zadania.

#### **ExecutorService**

Rozbudowana wersja interfejsu `Executor`, w projekcie zaimplementowana jako stała pula wątków (`Executors.newFixedThreadPool(4)`). Pula składa się ze stałej grupy przypisanych wątków, które wielokrotnie pobierają kolejne zadania z kolejki. Zapewnia to pełną kontrolę nad cyklem życia (np. ułatwione zatrzymywanie przez `shutdownNow()`). Mechanizm ten w dużym stopniu oszczędza zasoby komputera, ponieważ unika ciągłego tworzenia i niszczenia obiektów `Thread`.

#### **ScheduledExecutorService**

Rozszerza możliwości `ExecutorService` o wbudowany harmonogram. Zamiast wykonywać zadania od razu i w ciągłej pętli, pozwala na ich opóźnienie lub precyzyjne, cykliczne powtarzanie (`scheduleAtFixedRate`). W projekcie zastępuje to ręczne usypianie wątków (`Thread.sleep()`) oraz usuwa potrzebę stosowania pętli `while` w samym zadaniu, przenosząc logikę powtarzania na poziom zarządcy (frameworku).

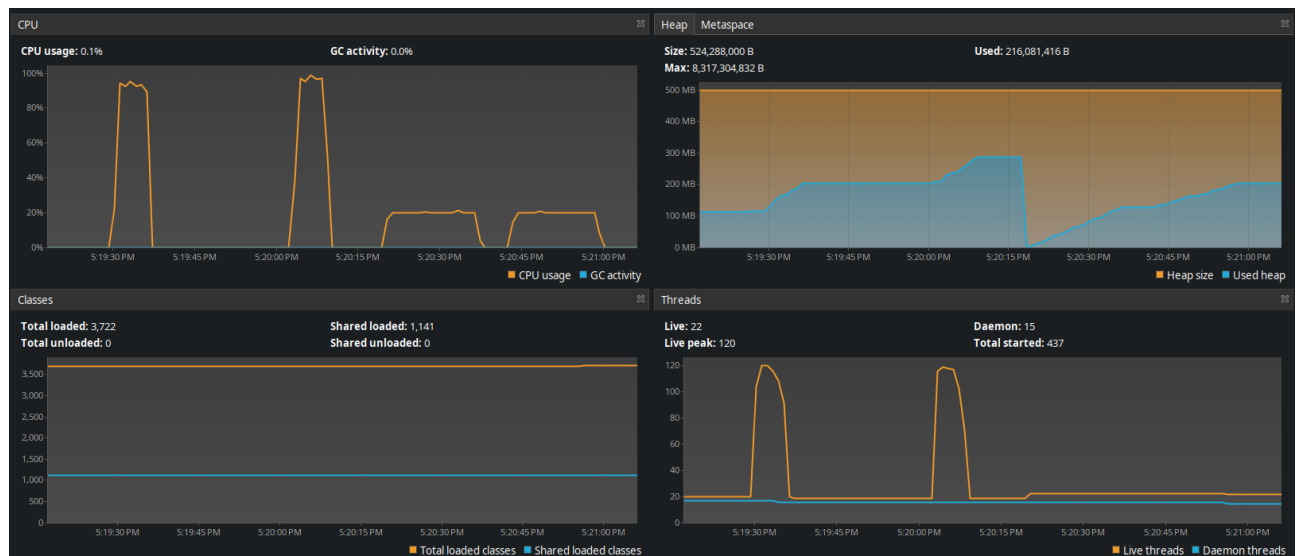
## 4. Analiza wyników i zachowania animacji

Obserwacja zachowania wizualnego klocków (reprezentujących wątki) w interfejsie graficznym doskonale odzwierciedla mechanikę działania poszczególnych podejść do wielowątkowości:

- **Thread + Runnable:** Wszystkie 100 klocków rusza jednocześnie, jednak ich ruch jest bardzo nieregularny, a wręcz losowy. Nie idą w partiach, ulegają rozproszeniu i stale przesuwiają się względem siebie. Wynika to z faktu, że planista systemu operacyjnego (OS scheduler) musi sprawiedliwie rozdzielić czas procesora między 100 aktywnych, ciężkich fizycznych wątków, co prowadzi do nierównomiernego dostępu do CPU.
- **Executor:** Przebieg animacji wygląda niemal identycznie jak w podejściu klasycznym. Wynika to z użytej implementacji interfejsu `Executor` (`command -> new Thread(command).start()`), która de facto zachowuje się pod spodem tak samo jak bezpośrednie użycie `Thread`.
- **ExecutorService (pula 4 wątków):** Klocki poruszają się wyraźnymi partiami (po 4 sztuki naraz). Wynika to z faktu, że pętla `while` w obiekcie `KrokAnimacji` „blokuje” wątek z puli aż do momentu, gdy klocek dotrze do mety. Dopiero po zakończeniu całego biegu klocka, wątek z puli jest zwalniany i pobiera z kolejki kolejne zadanie (kolejny klocek). Niewielkie przesunięcia wewnątrz poruszających się klocków wynikają z faktu, że wątki nadal muszą konkurować o zasoby systemowe.
- **ScheduledExecutorService:** Wszystkie klocki przemieszczają się niemal w jednej pionowej linii w sposób równomierny i falowy. To diametralnie inne zachowanie wynika ze zmiany struktury wywołania. Zamiast ciągłej pętli `while` dla każdego klocka, do planisty wrzucono 100 krótkich zadań (jeden skok o 2 piksele), które wykonują się co 10 milisekund z wykorzystaniem puli 4 wątków. Pula 4 wątków jest w stanie błyskawicznie obsłużyć 100 mikrozadań w ramach pojedynczego cyklu co 10ms, dzięki czemu klocki przesuwiają się synchronicznie.

## 5. Analiza z wykorzystaniem profilera

Na poniższym zrzucie ekranu zaprezentowano przebiegi analizujące wpływ działającego programu na zachowanie CPU oraz wątków.



Rys. 1: Analiza działania aplikacji z wykorzystaniem profilera

- **Technika klasyczna (Thread + Runnable):** Pierwszy skrajnie wysoki pik po lewej. Charakteryzuje się bardzo krótkim czasem wykonania (wąski słupek), ale ogromnym kosztem zasobów – zużycie CPU sięga blisko 100%, a liczba aktywnych wątków skacze do około 120 (tworzony jest nowy fizyczny wątek dla każdego klocka). Takie działanie powoduje, że klocki na ekranie przesuwają się nieregularnie i chaotycznie.
- **Podstawowy Executor:** Drugi wysoki pik na wykresie. Wykazuje parametry zużycia zasobów (CPU i skok liczby wątków) i czas wykonania w zasadzie identyczne jak podejście pierwsze. Każdorazowo tworzony jest nowy wątek. Odzwierciedla to animacja, która wizualnie zachowuje się tak samo niestabilnie jak przy metodzie klasycznej.
- **Podejście wysokopoziomowe (ExecutorService):** Trzeci, szerszy i znacznie niższy słupek. Widać tu drastyczną optymalizację: zużycie procesora spada i stabilizuje się (ok. 20%), a wykres wątków pozostaje płaski (system wykorzystuje tylko zadeklarowaną pulę 4 wątków roboczych). Czas całkowity wydłuża się, ponieważ zadania są systematycznie kolejkowane. Przekłada się to na animację, w której klocki idą partiami (po 4 sztuki).
- **Planowanie zadań (ScheduledExecutorService):** Czwarty, ostatni obszar po prawej. Pod kątem parametrów w profilerze (niskie CPU i płaska, minimalna liczba wątków z puli) wygląda równie optymalnie co **ExecutorService**. Kluczową różnicą jest tu wbudowane harmonogramowanie – wymuszanie regularnych wywołań w czasie pozwala zniwelować mikro-opóźnienia, dzięki czemu klocki na ekranie utrzymują się w niemal idealnej pionowej linii.

## 6. Wnioski końcowe

Eksperyment i analiza wyraźnie udowadniają wyższość wysokopoziomowego pakietu `java.util.concurrent` nad klasycznym podejściem. Użycie puli wątków (`ExecutorService`) pozwala kontrolować współbieżność (np. wymuszając obróbkę partiami), co stabilizuje użycie zasobów procesora.

Z kolei `ScheduledExecutorService` stanowi najlepsze rozwiązanie dla zadań cyklicznych (np. generowania klatek animacji), zapewniając synchronizację i odpowiednie zarządzanie czasem procesora bez blokowania wątków i stosowania niewydajnego `Thread.sleep()`.



## Spis rysunków

|   |                                                                  |   |
|---|------------------------------------------------------------------|---|
| 1 | Analiza działania aplikacji z wykorzystaniem profilera . . . . . | 7 |
|---|------------------------------------------------------------------|---|